

TABLA DE CONTENIDO

INTRODUCCIÓN	3
1. REVISION INICIAL	4
1.1. Estado Antes De Mejoras.....	4
2. ESTRUCTURA DE ARCHIVOS.....	6
2.1. Contact.Module.Ts.....	7
2.1.1. Necesidad de Mejora.....	7
2.1.2. Solución Implementada	8
2.2. Contac.Page.Ts.....	9
2.2.1. Mejora detectada	9
2.2.2. Solución implementada	9
2.2.3. Campos del FormGroup y sus validadores	10
2.2.4. Fragmentos importantes del código CONTAC .PAGE .TS11	
2.3. Contac.Page.Html	12
2.3.1. Mejora detectada	12
2.3.2. Solución implementada	12
2.3.3. Fragmentos de código: Enlaces y validaciones	13
2.4. Contac.Page.Scss.....	15
3. RESUMEN DE MEJORAS.....	16
4. INTERACTIVIDAD AVANZADA	16
4.1. Columnas De Contacto Directo	17
4.2. Formulario Ampliado.....	17
4.3. Rediseño Del Menu De Postres	19
5. DIFICULTADES Y SOLUCIONES	20
5.1. Calendario Con Días En Texto Garbled	20
5.2. Reset() Dejaba Numeropersonas En Null	21
6. EJECUCION Y VERIFICACION	21
7. RESUMEN DE ARCHIVOS MODIFICADOS	24

8. DISPONIBILIDAD DEL PROYECTO EN GITHUB	25
CONCLUSIONES	26
REFERENCIAS	27

INTRODUCCIÓN

El presente informe da continuidad a la actividad ejecutada en la Evaluación1, la cual tuvo por objetivo la construcción de una aplicación móvil híbrida proyecto denominado sweet-dreams-app, usando Ionic Framework y Angular. El proyecto incluyó las páginas de Inicio (home), información personal (personal-info), contacto (contact) y un carrito de compras (cart). En relación a la vista de contacto sobre la cual se enfoca el presente este informe se trabajó como un formulario básico.

En consecuencia, este informe documenta exclusivamente las mejoras aplicadas sobre la página de Contacto, sin modificar el resto de la aplicación (Home, Información, Personal, Producto-Detalle, Carrito), respecto a la entrega anterior. Este documento explicará todo el proceso paso a paso de migración de un formulario básico hacia formularios reactivos de Angular (Reactive Forms).

1. REVISION INICIAL

1.1. Estado Antes De Mejoras

Para llevar adelante esta actividad, en primer lugar, se realizó una revisión exhaustiva del código, donde se detectaron cuatro puntos de mejoras, los cuales deben ser modificados para que la pagina **contac** pase de ser un formulario básico a un formulario reactivo. Los cuales se mencionan a continuación:

- **Formulario sin Marcador:** Ausencia de la etiqueta `<form>`. Impedía a los campos **HTML** organizarse como unidad funcional, imposibilitando la captura y gestión de eventos nativos de envío (`submit`), debido a que los elementos carecían de un contexto jerárquico que los vinculara entre sí y con el componente.
- **No estaba implementada la directiva `ngModel` en los campos de entrada:** Aunque los controles (`<input>`, `<select>` y `<textarea>`), se mostraban de manera correcta en la interfaz y eran operativos estaban huérfanos de la lógica definida en **TypeScript**. Se podía interactuar con ellos e introducir datos, pero al no existir un enlace bidireccional que sincronizara el **DOM** con el modelo del componente, los valores ingresados nunca llegaban a las propiedades correspondientes; toda la información quedaba relegada al ámbito de la vista, sin repercusión alguna en el estado interno de la aplicación.
- **Botón *Solicitar Cotización* doble omisión funcional:** Ausencia de la directiva de enlace de eventos (`click`) y ausencia del atributo nativo `type="submit"` que le permite al navegador gestionar el envío estándar del formulario. Estas faltas de manera conjunta provocaban que al presionar el elemento no se vinculara a ninguna rutina de **TypeScript** y

mucho menos al mecanismo de entrega del formulario. Por lo que no desencadenaba acción alguna al presionar, dejando la interacción completamente inerte y sin efecto sobre la lógica de la aplicación.

- **El objeto contacto declarado en TypeScript definía una propiedad email que no encontraba correspondencia con ningún campo en la plantilla HTML.** A su vez, la interfaz mostraba controles para **tipoEvento**, **fechaEvento** y detalles, pero el modelo de datos carecía de propiedades equivalentes para almacenarlos. Esta doble desalineación —propiedades huérfanas sin representación visual y campos visibles sin destino en el modelo— generaba una ruptura total del principio de sincronización bidireccional. Como consecuencia, el componente no podía recibir los datos introducidos por el usuario, ni tampoco disponía de una estructura interna coherente para procesarlos, lo que invalidaba cualquier intento de validación, transformación o envío estructurado de la información.

2. ESTRUCTURA DE ARCHIVOS

Luego de establecer los puntos necesarios para aplicar las mejoras que permitan el logro de los objetivos planteados en esta actividad se muestra a continuación la estructura de archivos a modificar:

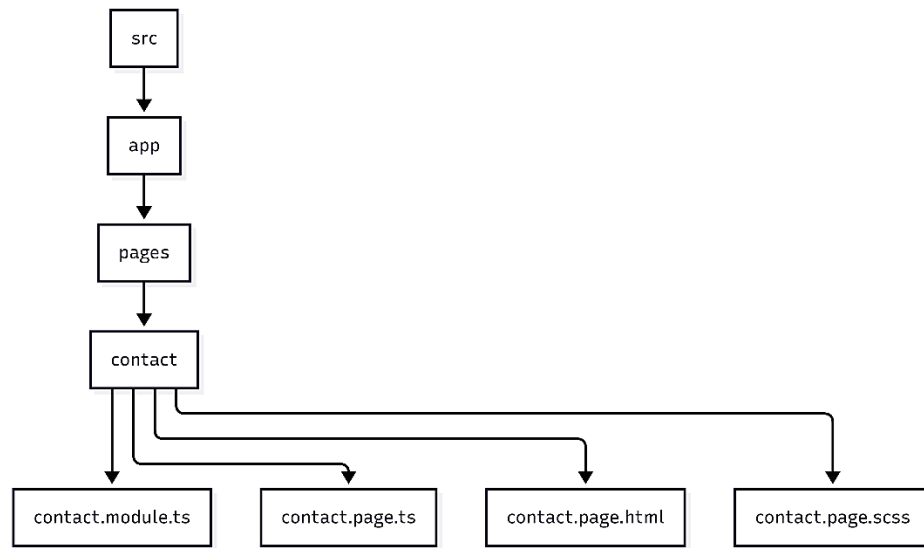


Figura 1 Estructura de directorios del módulo contact.

Fuente: Elaboración Propia (2026).

A continuación, se presenta una breve descripción del propósito de cada archivo, así como la naturaleza de las mejoras a implementar en cada caso. Cada paso responde a las dependencias jerárquicas del framework Angular, de manera que cada modificación sienta las bases para el siguiente paso garantizando la coherencia de las mejoras.

2.1. Contact.Module.Ts

2.1.1. Necesidad de Mejora

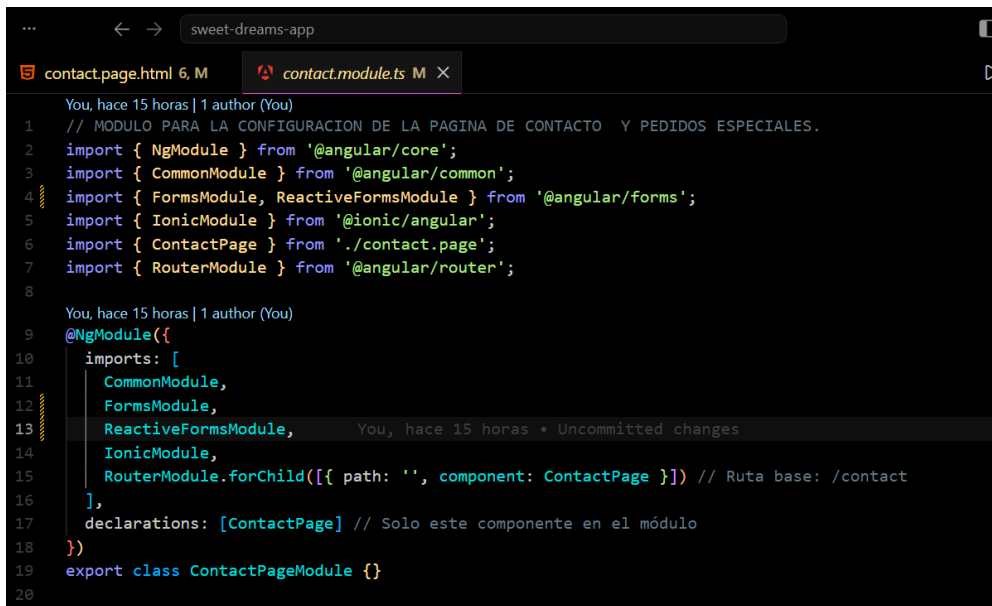
El **ContacModule** en su versión original solo tenía **FormsModule** entre sus importaciones, no se contempló declarar **ReactiveFormsModule**. Lo cual constituye un factor crítico debido a que **ReactiveFormsModule** es el conjunto que entrega las directivas estructurales **formGroup** y **formControlName**, elementos esenciales para enlazar el formulario reactivo delineado en el componente con su representación correspondiente dentro de la plantilla **HTML**.

Teniendo en cuenta que este módulo se carga a través de un mecanismo de carga diferida (**lazy loading**), la mera inclusión de **ReactiveFormsModule** en el módulo raíz **AppModule** resulta insuficiente para satisfacer sus requisitos. En Angular cada módulo de carga diferida representa un ámbito de compilación independiente y por lo tanto debe declarar explícitamente en su propiedad **imports** todas las dependencias necesarias para un óptimo funcionamiento. El pasar por alto esta declaración en el módulo correspondiente imposibilita al compilador para reconocer las directivas **formGroup** y **formControlName** durante el análisis de la plantilla, desencadenando un error de compilación que bloquea el proceso de construcción y, en consecuencia, el arranque (**bootstrapping**) de la aplicación.

El resultante de la ausencia de esas directivas en el contexto del módulo es un error de compilación ya que el compilador de Angular no consigue resolver de manera eficiente los enlaces de la plantilla. Esto impide por consiguiente la culminación del proceso de construcción (**build**) obstaculizando el arranque (**bootstrapping**) de la aplicación, impidiendo incluso la ejecución en el navegador.

2.1.2. Solución Implementada

Para resolver esta debilidad se incorporó **ReactiveFormsModule** en la propiedad **imports** del módulo **ContactPageModule** junto con **FormsModule**, **CommonModule** e **IonicModule**. De esta manera al residir la importación dentro del propio ámbito del módulo **lazy loading**, se garantiza que las directivas **formGroup** y **formControlName** estén disponibles en el contexto de compilación correspondiente. Esta configuración local satisface los requisitos del formulario reactivo sin generar dependencias cruzadas con el módulo principal, lo que permite que el compilador resuelva los enlaces de la plantilla sin errores y que la aplicación complete el proceso de construcción y arranque con normalidad.



```
...  < ->  sweet-dreams-app
contact.page.html 6, M  contact.module.ts M X
You, hace 15 horas | 1 author (You)
1  // MODULO PARA LA CONFIGURACION DE LA PAGINA DE CONTACTO Y PEDIDOS ESPECIALES.
2  import { NgModule } from '@angular/core';
3  import { CommonModule } from '@angular/common';
4  import { FormsModule, ReactiveFormsModule } from '@angular/forms';
5  import { IonicModule } from '@ionic/angular';
6  import { ContactPage } from './contact.page';
7  import { RouterModule } from '@angular/router';
8
9  @NgModule({
10   imports: [
11     CommonModule,
12     FormsModule,
13     ReactiveFormsModule,
14     IonicModule,
15     RouterModule.forChild([{ path: '', component: ContactPage }]) // Ruta base: /contact
16   ],
17   declarations: [ContactPage] // Solo este componente en el módulo
18 })
19 export class ContactPageModule {}
20
```

Figura 2 *contact.module.ts* con **ReactiveFormsModule** importado correctamente

Fuente: *Elaboración Propia (2026).*

2.2. Contac.Page.Ts

2.2.1. Mejora detectada

En el archivo original del componente, la lógica de datos estaba basada en un objeto estático denominado `contacto`. Este objeto declaraba una propiedad `email` que carecía de correspondencia con cualquier campo de la plantilla HTML, mientras que los campos que sí estaban visibles en la interfaz (`tipoEvento`, `fechaEvento` y `detalles`) no tenían propiedades equivalentes dentro de dicho objeto. Esta discrepancia entre el modelo y la vista generaba una ruptura del principio de sincronización de datos, impidiendo que el componente pudiera recibir o procesar la información introducida por el usuario. De igual manera este componente no contaba con un `FormGroup` que permitiera agrupar los controles del formulario y gestionara su estado, validación e interacción. Se carecía de un método para el envío de datos; la pulsación del botón no desencadenaba acción alguna sobre la lógica `TypeScrip`.

2.2.2. Solución implementada

La manera eficiente de resolver lo antes expuesto fue la sustitución del objeto `contacto` por un formulario reactivo declarado mediante **`FormBuilder`**. A través de **`this.fb.group({})`**, se contruyó un **`FormGroup`** que integra los campos que se muestran en la interfaz: **`nombre`**, **`tipoEvento`**, **`fechaEvento`**, **`numeroPersonas`** y **`detalles`**. Para cada control se tomó la decisión del asignarle validadores específicos (**`Validators.required`**, **`minLength`**, **`maxLength`** y **`min`**) esto es garantía de integridad en los datos.

También se implementó el método **`enviarFormulario()`**, el cual evalúa la validez del formulario, construye un objeto completo con todos los datos (incluyendo los estados locales como la selección de postres y la imagen de referencia), muestra notificaciones mediante **`ToastController`**, y reinicia el formulario tras una solicitud exitosa.

Esta arquitectura reactiva establece un flujo unidireccional y controlado entre la vista y el modelo, resolviendo la desconexión previa y permitiendo un manejo robusto de la interacción del usuario.

2.2.3. Campos del FormGroup y sus validadores

Como parte de la implementación del formulario reactivo, se definieron validadores específicos para garantizar la integridad de los datos antes del envío. La **tabla 1** sintetiza esta configuración, detallando el tipo de campo, las restricciones impuestas y los valores por defecto establecidos para cada control del FormGroup.

Tabla 1 Controles del FormGroup y sus validadores

Control	Tipo de campo	Validadores aplicados
Nombre	ion-input (texto libre)	required + minLength(3)
tipoEvento	ion-select (lista fija)	required (no aplica minLength en selects)
fechaEvento	ion-datetime en popover	required (valor asignado por patchValue)
numeroPersonas	Contador personalizado	required + min(1), valor inicial: 15
detalles	ion-textarea (texto largo)	required + minLength(10) + maxLength(500)

Fuente: *Elaboración Propia (2026).*

2.2.4. Fragmentos importantes del código CONTAC.PAGE.TS

A continuación, se muestra el extracto del método `ngOnInit()` donde se instancia el **FormGroup** con los cinco controles y sus respectivos validadores. Este bloque constituye la evidencia directa de la configuración resumida en la tabla anterior.



```
1 // =====
2 // LÓGICA DE LA PÁGINA DE PEDIDOS ESPECIALES – SWEET DREAMS
3 // Se aplica mejoras de Formularios Reactivos (ReactiveFormsModule).
4 // para mayor interactividad de la pagina.
5 // =====
6
7 import { Component, OnInit } from '@angular/core';
8
9 // FormBuilder: Este servicio simplifica la creación del FormGroup
10 // FormGroup: Desde este contenedor agrupamos todos los controles del formulario
11 // Validators: Funciones definidas de validación (required, minLength, etc.)
12 import { FormBuilder, FormGroup, Validators } from '@angular/forms';
13
14 // ToastController: servicio de Ionic para mostrar notificaciones emergentes
15 import { ToastController } from '@ionic/angular';
16
17 constructor(
18   private fb: FormBuilder, // Aquí se inyecta FormBuilder
19   private toastCtrl: ToastController // Aca se inyecta ToastController
20 ) {}
21
22 ngOnInit() {
23   this.contactForm = this.fb.group({
24
25     // Nombre: Obligatorio, mínimo 3 caracteres y texto libre.
26     nombre: ['', [Validators.required, Validators.minLength(3)]],
27
28     // Tipo de evento: Selector obligatorio (se debe elegir una opción).
29     tipoEvento: ['', Validators.required],
30
31     // Fecha del evento: Obligatorio, se asigna mediante el popover de ion-datetime.
32     fechaEvento: ['', Validators.required],
33
34     // numeroPersonas: número entero, obligatorio, mínimo 1
35     // Valor inicial: 1 (valor por defecto sugerido)
36     numeroPersonas: [1, [Validators.required, Validators.min(1)]],
37
38     // Detalles: texto entre 10 y 500 caracteres
39     detalles: ['', [
40       Validators.required,
41       Validators.minLength(10),
42       Validators.maxLength(500)
43     ]],
44   });
45 }
```

Figura 3 Extracto del archivo `contact.page.ts`

Fuente: Elaboración Propia (2026)

2.3. Contac.Page.Html

2.3.1. Mejora detectada

En la versión anterior de la plantilla, los campos del formulario carecían de una etiqueta contenedora `<form>` que los agrupara funcionalmente, y no existía ningún mecanismo de enlace que conectara los datos introducidos por el usuario con la lógica del componente. El botón de envío, además, no contaba con el atributo `type="submit"` ni con ningún manejador de eventos, por lo que su pulsación no desencadenaba acción alguna.

2.3.2. Solución implementada

Con una estructura sólida en TypeScript mediante el **FormGroup**, se procedió a la actualización del **HTML** para hacer la conexión con el modelo reactivo. Los cambios se aplicaron exclusivamente a la columna derecha del formulario; el encabezado, el bloque hero y la columna de redes sociales mantuvieron su estructura previa sin modificaciones.

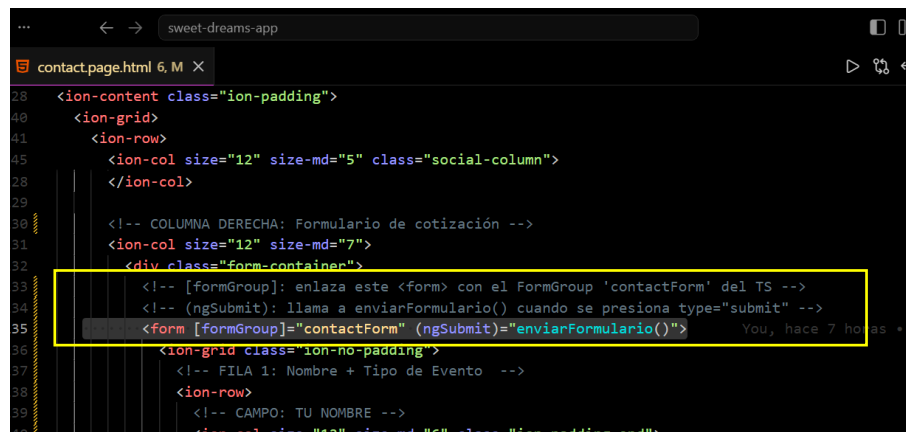
Las correcciones aplicadas fueron las siguientes:

- **Contenedor del formulario:** Se envolvió todo el contenido en una etiqueta `<form [formGroup]="contactForm" (ngSubmit)="enviarFormulario()">`, elemento que antes no existía.
- **Enlace de datos:** Cada campo (`ion-input`, `ion-select`, `ion-textarea`) recibió su directiva `formControlName` correspondiente, reemplazando la ausencia total de enlace previa.
- **Mensajes de error:** Se incorporó un bloque `<div class="error-msg">` debajo de cada campo, con condiciones `*ngIf` que los hacen visibles únicamente cuando el campo ha sido tocado (`touched`) o cuando se ha intentado enviar el formulario (`submitted`).

- **Botón de envío:** Se asignó explícitamente el atributo `type="submit"` al botón, garantizando que su pulsación active correctamente el evento (`ngSubmit`) y ejecute el método `enviarFormulario()`.

2.3.3. Fragmentos de código: Enlaces y validaciones

Los siguientes extractos del archivo `contact.page.html` evidencian la implementación de las correcciones descritas. Se incluyen el contenedor del formulario reactivo, el enlace de los campos mediante `formControlName`, la estructura de mensajes de error condicionados a la interacción del usuario, y el botón de envío configurado con `type="submit"`. Estos fragmentos demuestran cómo se conecta la plantilla con el `FormGroup` definido en el `TypeScript`, garantizando la validación y el flujo de datos del formulario.



```

28 <ion-content class="ion-padding">
40   <ion-grid>
41     <ion-row>
45       <ion-col size="12" size-md="5" class="social-column">
46       </ion-col>
47     </ion-row>
48     <!-- COLUMNA DERECHA: Formulario de cotización -->
49     <ion-col size="12" size-md="7">
50       <div class="form-container">
51         <!-- [formGroup]: enlaza este <form> con el FormGroup 'contactForm' del TS -->
52         <!-- (ngSubmit): llama a enviarFormulario() cuando se presiona type="submit" -->
53         <form [formGroup]="contactForm" (ngSubmit)="enviarFormulario()">
54           <ion-grid class="ion-no-padding">
55             <!-- FILA 1: Nombre + Tipo de Evento -->
56             <ion-row>
57               <!-- CAMPO: TU NOMBRE -->
58               <ion-col size="12" size-md="6" class="ion-padding-end">

```

Figura 4 Etiqueta `<form>` con enlace al `FormGroup` y evento de envío

```

Ejecutar ...  ← →  sweet-dreams-app
contact.page.html 6, M X
28 <ion-content class="ion-padding">
40   <ion-grid>
41     <ion-row>
131       <ion-col size="12" size-md="7">
132         <div class="form-container">
138           <ion-row>
139             <!-- CAMPO: TU NOMBRE -->
140             <ion-col size="12" size-md="6" class="ion-padding-end">
141               <ion-item
142                 lines="none"
143                 class="sweet-input box-input" You, hace 7 horas • Uncommitted changes
144                 [class.input-error]="submitted && controles['nombre'].invalid"
145                 [class.input-valid]="controles['nombre'].valid"
146               >
147                 <ion-label position="stacked">Tu Nombre completo</ion-label>
148                 <!-- formControlName enlaza este input con el control 'nombre' del FormGroup -->
149                 <ion-input
150                   type="text"
151                   formControlName="nombre"
152                   placeholder="¿Cómo te llamas?"
153                 >
154               </ion-input>
155             </ion-item>
156             <!-- Mensajes de error: visibles si el campo fue tocado o se intentó enviar -->
157             <div
158               class="error-msg"
159               *ngIf="(submitted || controles['nombre'].touched) && controles['nombre'].errors"
160             >
161               <span *ngIf="controles['nombre'].errors?.['required']">
162                 >Tu nombre es necesario.</span>
163               <span *ngIf="controles['nombre'].errors?.['minlength']">
164                 >Mínimo 3 caracteres.</span>
165             </div>
166           </ion-col>
167         </div>
168       </ion-col>

```

Figura 5 Campo nombre con formControlName y mensajes de error

Fuente: Elaboración Propia (2026)

```

>
<!-- formControlName sobre ion-select: Angular gestiona el valor seleccionado -->
<ion-select
  placeholder="Selecciona un evento"
  formControlName="tipoEvento"
>
  <ion-select-option value="boda">
    >Boda / Matrimonio</ion-select-option>
  >
  <ion-select-option value="cumple">
    >Cumpleaños</ion-select-option>
  >
  <ion-select-option value="bautizo">
    >Bautizo / Primera Comunión</ion-select-option>
  >
  <ion-select-option value="corporativo">
    >Evento Corporativo</ion-select-option>
  >
  <ion-select-option value="otro">
    >Otro Momento Especial</ion-select-option>
  >
</ion-select>

```

Figura 6 Campo tipoEvento con ion-select y formControlName

Fuente: Elaboración Propia (2026)

```

462
463 <!-- BOTÓN PRINCIPAL DE ENVÍO -->
464 <!-- type="submit" es obligatorio para que (ngSubmit) se dispare -->
465 <!-- expand="block" lo hace ancho completo -->
466 <div class="ion-padding-top ion-margin-top">
467   <ion-button
468     expand="block"
469     shape="round"
470     color="primary"
471     class="send-btn"
472     type="submit"
473   >
474     SOLICITAR COTIZACIÓN
475   </ion-button>
476 </div>
477 </form>
478 </div>
479 </ion-col>
480 </ion-row>
481 </ion-grid>
482 </ion-content>
483

```

Figura 7 Botón de envío con type="submit"

Fuente: *Elaboración Propia (2026)*

2.4. Contac.Page.Scss

Para complementar la validación funcional implementada en el formulario reactivo, se añadieron al archivo **contact.page.scss** las siguientes clases de estilo, sin modificar los estilos preexistentes:

.input-error y .input-valid: aplican un borde de color rojo (estado inválido) o verde (estado válido) a los campos del formulario, proporcionando una señal visual clara del estado de cada control.

.error-msg: unifica la presentación de todos los mensajes de error, con un tamaño de fuente, color y espaciado consistentes en toda la página.

Estas adiciones estéticas refuerzan la experiencia del usuario al ofrecer retroalimentación visual inmediata y coherente con los mensajes de error condicionales definidos en la plantilla.

3. RESUMEN DE MEJORAS

La siguiente tabla muestra un comparativo de los principales aspectos funcionales y visuales entre la versión inicial de la página y las mejoras aplicadas.

Tabla 2 Comparativa de mejoras aplicadas en la página de contacto, abarcando desde el enlace de datos y validación hasta las mejoras visuales y de usabilidad.

Aspecto	Estado real antes	Estado después del ajuste
Enlace de datos	Inexistente (sin ngModel)	formControlName en cada campo
Envío del formulario	Botón sin acción	(ngSubmit) ejecuta enviarFormulario()
Validación	No existía en la vista	Validators.required, minLength, maxLength
Mensajes de error	No existían	Contextuales por campo con *ngIf
Botón de correo	No ejecutaba ninguna acción	window.open('mailto:...') funcional
Calendario	Días en inglés / garbled	locale='es-ES', semana inicia en lunes
Campos en pantalla	3 (nombre, tipoEvento, detalles)	6 (+ fecha, personas, postres, imagen)
Botones sociales	Estáticos, sin acción	Abren WhatsApp, IG, correo, Google Maps
Menú de postres	Chips básicos sin icono	Tarjetas visuales con emoji, check, animación

Fuente: *Elaboración Propia (2026)*

4. INTERACTIVIDAD AVANZADA

Una vez completada la migración a formularios reactivos, se identificó que el nivel de interactividad podía elevarse significativamente más allá de la validación de campos. Esta segunda fase se abordó tanto la columna de contacto directo como la profundidad de información que el formulario es capaz de capturar en una sola interacción con el usuario.

4.1. Columnas De Contacto Directo

En la versión original, los ítems: Instagram, WhatsApp, correo electrónico y ubicación, se constituían elementos informativos (icono y Texto) sin evento asociado. Esta es precisamente la diferencia entre una interfaz que parece interactiva y una que realmente lo es.

Para escalar esta mejora se programó un método en TypeScript que abre el servicio correspondiente fuera de la aplicación, respetando el flujo nativo de cada plataforma

Para cada ítem se programó un método en el TypeScript que abre el servicio correspondiente fuera de la aplicación, respetando el flujo nativo de cada plataforma

```
// GETTER DE CONTROLES
// Permite acceder a los campos en el HTML de forma más corta,
// omitiendo el nombre del formulario.
get controles() {
    return this.contactForm.controls;
}

// MÉTODOS DE CONTACTO DIRECTO

abrirInstagram() {
    // _blank: abre en una nueva pestaña del navegador
    window.open('https://instagram.com/SweetDreams', '_blank');
}

abrirWhatsApp() {
    // wa.me genera un enlace directo a WhatsApp Web o la app instalada
    window.open('https://wa.me/584241234567', '_blank');
}

// window.open abre el correo sin bloquear la navegación actual de la app.
abrirCorreo() {
    window.open('mailto:contacto@gmail.com');
}

abrirMapa() {
    // Abre Google Maps centrado en Caracas, Venezuela
    window.open('https://maps.google.com/?q=Caracas+Venezuela', '_blank');
}
```

Figura 8 Extracto del archivo contact.page.ts que implementa los métodos `abrirInstagram()`, `abrirWhatsApp()`, `abrirCorreo()` y `abrirMapa()`.

4.2. Formulario Ampliado

En busca de potenciar la experiencia usuario se sometió a escrutinio la capacidad que tenía el formulario de poder absorber la mayor cantidad de datos que pudiesen transformar una consulta o solicitud de presupuesto puntual en una venta, reduciendo la necesidad de contactos adicionales. El

formulario original con tres campos (nombre, tipo de evento, detalles) era insuficiente para ese objetivo. El formulario final tiene seis campos que cubren las dimensiones esenciales de cualquier pedido:

Tabla 3 Campos del formulario y su propósito en la captación de información

Campo	Información que captura	Decisión de diseño
Nombre	¿Quién hace el pedido?	Texto libre, mínimo 3 caracteres para evitar iniciales sueltas
Tipo de Celebración	¿Qué tipo de evento?	ion-select con 5 opciones; la categoría permite pre-calcular rangos de precio
Fecha del Evento	¿Cuándo es el evento?	ion-datetime con locale es-ES; evita fechas pasadas con [min]=today
Cantidad de Personas	¿Cuántos invitados?	Contador +- más intuitivo que un input numérico; determina el tamaño del pedido
Menú de Postres	¿Qué tipo de postres?	Selección múltiple visual; tarjetas con emoji, nombre y checkmark animado
Imagen de Referencia	¿Tiene diseño en mente?	Upload opcional con vista previa; reduce malentendidos sobre la decoración
Detalles	¿Hay especificaciones?	Textarea con contador de 500 caracteres; captura alergias, colores, temática

Fuente: Elaboración Propia (2026)

Pedidos Especiales

Cotiza tu pedido especial
Postres personalizados para cada celebración.

yContáctanos!

Haz clic en cualquiera de estas opciones para comunicarte con nosotros de inmediato.

Instagram

@SweetDreams

WhatsApp Directo

+58 424-1234567

Correo Electrónico

contacto@sweetdreams.com

Nuestra Tienda

Cumaná, Venezuela (Ver Mapa)

De cuántos cumple?

¿Cómo te llamas?

Selecciona un evento ▼

Elige la fecha

Elige el sabor

1

Mundo de Postres de Interior
Selecciona las variaciones para tu celebración (3 elegidas)

Tortas Tradicionales

Tortas Modernas

Mosaico de Sabores

Cupcakes

Galletitas

Pastelería Fine

Imagen de Referencia (Opcional)

Toma apñ para subir la imagen de referencia
(JPG o PNG, Máximo 5 MB)

Instrucciones y Notas
Especifica sabores, alergias, colores o temáticas de la decoración.

0000

SOLICITAR COTIZACIÓN

Figura 9 Formulario completo con los 6 campos visibles en pantalla

Fuente: Elaboración Propia (2026)

4.3. Rediseño Del Menu De Postres

Este menú resultaba muy básico, solo manejaba texto y alternaba algo de color al seleccionarse sin retroalimentación visual suficiente para poder transmitir cual se encontraba activo. El rediseño reemplazó los chips por tarjetas (**postre-card**) organizadas en cuadrícula con tres elementos visuales: un emoji identificador del tipo de postre, el nombre, y un ícono **checkmark/círculo** que cambia de estado al seleccionar. La transición de color al estado primario y la animación de elevación confirman visualmente la selección de forma inequívoca.



```
export class ContactPage implements OnInit {
  decrementarPersonas() {
    if (val > 1) {
      this.contactForm.patchValue({ numeroPersonas: val - 1 });
    }
  }

  // -- SELECCIÓN MÚLTIPLE DE POSTRES --

  // togglePostre: agrega o quita un postre de la lista de seleccionados.
  // Si ya existe lo elimina; si no, lo agrega.
  togglePostre(postre: string) {
    const index = this.postresSeleccionados.indexOf(postre);
    if (index > -1) {
      this.postresSeleccionados.splice(index, 1); // Quitar
    } else {
      this.postresSeleccionados.push(postre); // Agregar
    }
  }

  // isPostreSelected: retorna true si el postre ya está en la selección.
  // Usado en el HTML para alternar el color y el ícono del chip.
  isPostreSelected(postre: string): boolean {
    return this.postresSeleccionados.includes(postre);
  }

  // getPostreIcono: retorna el emoji del mapa postresIconos.
  // Devuelve un valor por defecto, si no se encuentra el resultado.
  getPostreIcono(postre: string): string {
    return this.postresIconos[postre] || '🍰';
  }

  // -- GESTIÓN DE IMAGEN DE REFERENCIA --

  // onFileSelected: se ejecuta cuando el usuario elige un archivo en el input[type=file].
  onFileSelected(event: any) {
    const file = event.target.files[0];
    if (file) {
      this.selectedFile = file;
      // FileReader lee el archivo localmente (sin subirlo a ningún servidor aún)
      // y genera una URL basada que puede usarse directamente en un 
      const reader = new FileReader();
      reader.onload = () => {
        this.filePreview = reader.result;
      };
      reader.readAsDataURL(file);
    }
  }
}
```

Figura 10 Fragmento de código del Menú de Postres

Fuente: Elaboración Propia

Figura 11 Vista contacto Menú selección Interactiva

Fuente: Elaboración Propia (2026)

5. DIFICULTADES Y SOLUCIONES

Durante el proceso de migración y mejora se presentaron varios errores, algunos en tiempo de compilación y otros solo detectables al ejecutar la aplicación. Se documentan a continuación con el mensaje exacto, la causa técnica y la solución aplicada, ya que estas dificultades forman parte del aprendizaje real y demuestran comprensión del funcionamiento interno del framework.

5.1. Calendario Con Días En Texto Garbled

Al ejecutar la aplicación y abrir el selector de fecha, el calendario mostraba los nombres de los días de la semana como caracteres sin sentido o como letras sueltas ('D', 'L', 'M', 'X') que en algunos navegadores o sistemas aparecían cortadas o seguidas de texto ilegible. El mes y el año también tenían un tamaño desproporcionado respecto al contenedor del popover.

Causa: el componente `ion-datetime`, cuando no recibe un atributo `locale`, utiliza el idioma del navegador en el que se ejecuta. En entornos de desarrollo sin `locale` configurado explícitamente, esto puede producir nombres de días inconsistentes o abreviados de forma incorrecta. Adicionalmente, sin un ancho fijo en el popover (`.date-popover`), el calendario intentaba adaptarse a su contenedor y los textos del encabezado de mes/año se desbordaban.

5.2. `Reset()` Dejaba `Numeropersonas` En Null

Después del envío exitoso del formulario, `contactForm.reset()` limpiaba todos los campos correctamente excepto uno: el contador de personas mostraba null en lugar de su valor por defecto de 15, causando que el contador apareciera en blanco en un segundo uso del formulario.

Causa: `reset()` sin parámetros establece todos los controles a null, ignorando los valores iniciales definidos en el `fb.group()`. Esto es un comportamiento documentado de Angular: el valor inicial que se pasa al `FormControl` en la construcción del `FormGroup` no es un valor por defecto persistente, sino solo el valor inicial en la primera carga.

6. EJECUCION Y VERIFICACION

Luego de realizadas las mejoras y correcciones documentadas en las secciones anteriores, se llevaron a cabo las pruebas funcionales sobre el formulario de contacto. Con el fin de verificar que cada uno de los aspectos abordados operara conforme a lo esperado en un entorno de ejecución real.

Las pruebas se diseñaron para cubrir tanto los flujos principales (envío exitoso del formulario) como los casos límite (campos vacíos, validaciones,

interacciones con elementos auxiliares). Cada prueba se ejecutó manualmente sobre la aplicación en funcionamiento, simulando el comportamiento de un usuario final.

La siguiente tabla resume los casos de prueba ejecutados, el resultado esperado y el estado de verificación obtenido:

Tabla 4 Pruebas realizadas

#	Prueba	Resultado esperado	Estado
1	Enviar con todos los campos vacíos	Mensajes de error en todos los campos + toast amarillo	Correcto
2	Tocar Instagram	Abre instagram.com en nueva pestaña	Correcto
3	Tocar WhatsApp	Abre wa.me con el número configurado	Correcto
4	Tocar Correo	Abre el cliente de correo del sistema	Correcto
5	Tocar el campo de fecha	Popover con calendario en español, lunes al inicio	Correcto
6	Escribir 'Ab' en Nombre	Mensaje 'Mínimo 3 caracteres' al salir del campo	Correcto
7	Seleccionar postres en el menú	Tarjetas cambian a color primario + checkmark	Correcto
8	Posicionar cursor en zona de imagen	El cursor cambia a manito	Correcto
9	Subir una imagen	Vista previa con botón Quitar imagen	Correcto
10	Llenar todos los campos y enviar	Toast verde + formulario limpio (personas vuelve a 1)	Correcto

Fuente: Elaboración Propia (2026)

Cotiza tu pedido especial
Podrás añadirlos personalizados para cada celebración.

¡Contáctanos!
Has clic en cualquiera de estas opciones para comunicarte con nosotros de inmediato.

- Instagram @SweetDreams
- WhatsApp Directo 238 424 12487
- Correo Electrónico contacto@sweetdreams.com
- Nuestra Tienda Caracas, Venezuela (Ver Mapa)

Formulario de Cotización:

Nombre Completo: [Campo de texto]

Correo Electrónico: [Campo de texto]

Fecha de Evento: [Campo de fecha: 28 de junio de 2026]

Cantidad de Invitados: [Campo de número: 9]

Menú de Pastelería de Interés: [Campo de texto]

Imagen de Referencia: [Campo de imagen]

Detalles adicionales o Temática: [Campo de texto]

SOLICITAR COTIZACIÓN

Figura 12 Formulario con todos los campos válidos completos antes del envío fina

Fuente: Elaboración Propia (2026)

Pedidos Especiales

Variables: Vinculación (Ver Mapa)

Confirmación: ¡Solicitud enviada con éxito! Te contactaremos pronto.

Opciones de Pastelería:

- Cupcakes
- Galletas
- Pastelería Fina

Imagen de Referencia (Opcional):

Toca aquí para subir tu imagen de referencia
JPG o PNG - Máximo 5 MB

Detalles adicionales o Temática:
Especifica sabores, alergias, colores o temática de la decoración...

SOLICITAR COTIZACIÓN

Figura 13 Toast verde de confirmación '¡Solicitud enviada con éxito!

Fuente: Elaboración Propia (2026)

7. RESUMEN DE ARCHIVOS MODIFICADOS

Tabla 5 Tabla Resumen Archivos Modificados

Archivo	Qué se modificó	Por qué fue necesario
contact.module.ts	Importar ReactiveFormsModule	Sin este import, formGroup y formControlName no son reconocidos por Angular
contact.page.ts	Reemplazar objeto contacto por FormGroup de 5 controles; agregar métodos abrirInstagram, abrirWhatsApp, abrirCorreo (fix), abrirMapa, onChange, formatearFecha, toggle/isPostreSelected, getPostreIcono, onFileSelected, removerImagen	El modelo debe coincidir con los campos reales del HTML; los métodos convierten los botones estáticos en acciones reales
contact.page.html	Agregar <form [formGroup]> y (ngSubmit); formControlName en cada campo; locale en ion-datetime; redesign del menú de postres; reestructuración de la zona de imagen; type='submit' en el botón	Conecta la vista con el FormGroup, corrige el calendario y mejora la UX de todos los elementos interactivos
contact.page.scss	Agregar .box-input, .input-error, .input-valid, .error-msg, .seccion-postres, .postres-grid, .postre-card, .upload-zone (cursor+texto), .date-popover, .detalles-footer	Los estilos soportan la retroalimentación visual (bordes de color) y corrigen las proporciones tipográficas en toda la página

Fuente: Elaboración Propia (2026)

8. DISPONIBILIDAD DEL PROYECTO EN GITHUB

El proyecto completo está disponible en un repositorio público de github, lo que facilita su acceso y revisión. El desarrollo del código fuente conto con el apoyo de herramientas de inteligencia artificial generativas, específicamente claude ai y deepseek. El código resultante fue revisado, validado e integrado manualmente por la desarrolladora del proyecto. Pueden acceder mediante el siguiente enlace: <https://github.com/icoronado48-tech/sweet-dreams-app.git>

CONCLUSIONES

La evaluación sobre la página de contacto permitió superar un estado inicial carente de enlace entre vista y modelo, así como de lógica asociada a los eventos de envío. El resultado es un componente plenamente funcional que habilita al usuario para comunicarse a través de cuatro canales distintos y cursar solicitudes de cotización con todos los datos necesarios para su procesamiento.

La migración a formularios reactivos elevó cualitativamente la experiencia de usuario: el formulario ofrece retroalimentación visual instantánea (bordes rojos/verdes) y mensajes de error específicos para cada tipo de infracción, mientras que el envío ejecuta una validación robusta que sustituye la precaria verificación imperativa previa con sentencias if/else y alertas nativas.

El valor formativo del proceso residió en la resolución de problemas concretos en un entorno real con Angular e Ionic. La corrección de errores consolidó una comprensión aplicada del framework que trasciende el conocimiento teórico. Esta experiencia ha fortalecido la autonomía, el criterio técnico y la capacidad de depuración en entornos híbridos, competencias esenciales para el desarrollo profesional en aplicaciones multiplataforma.

REFERENCIAS

Angular Team. (2024). Reactive forms — Angular Documentation. Recuperado de <https://angular.io/guide/reactive-forms>

Angular Team. (2024). AbstractControl API. Recuperado de <https://angular.io/api/forms/AbstractControl>

Angular Team. (2024). NG01350 — formControlName must be used with a parent formGroup. Recuperado de <https://angular.io/errors/NG01350>

Anthropic. (2026). Claude AI (Versión Sonnet 4.6). <https://claude.ai/>
Ionic Framework. (2024). ion-datetime — Ionic Documentation. Recuperado de <https://ionicframework.com/docs/api/datetime>

Ionic Framework. (2024). ion-select — Ionic Documentation. Recuperado de <https://ionicframework.com/docs/api/select>

MDN Web Docs. (2024). window.open() — Web APIs. Recuperado de <https://developer.mozilla.org/en-US/docs/Web/API/Window/open>

Video de apoyo 1 (2025). Formularios reactivos en Angular con Ionic. YouTube. <https://www.youtube.com/watch?v=MMRqyQVQ980>

Video de apoyo 2 (2025). Validaciones con Angular Reactive Forms. YouTube. https://www.youtube.com/watch?v=bJ_DALcR1Ts